**King Fahd University of Petroleum & Minerals**

Department of Aerospace Engineering

# RoboMaster Assessment 1: Teleoperation, Telemetry, and ROS 2 Integration

**Course:** SCE 672 – Advanced Robotics **Instructor:** Prof. Aamer Iqbal Bhatti
**Student Name:** Raian E Rabbani **Student ID:** 202617880
**Department:** Aerospace Engineering **Date:** September 2026

## Problem 1: System Setup and Driver Installation

### (i) Operating System Setup

Instead of dual-booting Ubuntu, I set up my environment using **WSL 2 (Ubuntu 24.04)** on Windows. The handout advises against using virtual machines due to USB latency and dropped UDP packets, but WSL 2 handled the robot connections cleanly without these issues.

Here is how I verified that both connection methods work:

- **Wired USB (RNDIS):** I attached the RoboMaster's USB network device to WSL with `usbipd wsl attach`. I verified the connection by pinging `192.168.42.2`, getting consistent $< 1$ ms replies with no packet loss.
- **Wi-Fi Hotspot (AP):** I enabled mirrored networking in `.wslconfig` so WSL shares the host Wi-Fi adapter directly. Once connected to the robot's AP network, I verified that WSL pings `192.168.12.1` directly and runs the Python SDK without socket errors.

### (ii) ROS 2 and Simulation Setup

Inside Ubuntu on WSL 2, I installed the ROS 2 Jazzy desktop package along with Gazebo Harmonic and development tools:

```
$ sudo apt update && sudo apt install -y ros-jazzy-desktop-full \
    ros-jazzy-ros-gz python3-colcon-common-extensions libopus-dev \
    python3-pip ros-jazzy-teleop-twist-keyboard ros-jazzy-tf2-tools
```

### (iii) Python Virtual Environment

Ubuntu 24.04 enforces PEP 668, blocking global `pip install` commands. However, our scripts and ROS nodes need access to system-installed packages like `rclpy`.

To handle this cleanly, I created a virtual environment with the `--system-site-packages` option and installed the required SDK libraries inside it:

```
$ python3 -m venv --system-site-packages ~/rm_venv
$ source ~/rm_venv/bin/activate
$ python -m pip install -i https://test.pypi.org/simple/ rm-libmedia-codec
$ python -m pip install "numpy<2" "numpy-quaternion" pyyaml
$ python -m pip install git+https://github.com/jeguzzi/RoboMaster-SDK.git
```

### (iv) Building the ROS 2 Driver

I cloned the `robomaster_ros` driver package into a clean workspace and built it with `colcon`:

```
$ mkdir -p ~/ros2_ws/src && cd ~/ros2_ws/src
$ git clone https://github.com/jeguzzi/robomaster_ros.git
$ cd ~/ros2_ws && colcon build --symlink-install
```

### (v) NumPy 2.x ABI Issue and Diagnostic Checks

**The NumPy 2.x Conflict:** NumPy 2.0 introduced major breaking changes to its internal C API and memory layouts. In ROS 2 Jazzy on Ubuntu 24.04, binary packages like `cv_bridge` are precompiled against NumPy 1.x C headers. If NumPy gets updated to 2.x, importing `cv_bridge` causes memory corruption and crashes with an import error or segfault:

```
ImportError: numpy.core.multiarray failed to import
RuntimeError: module compiled against API version 0x10 but this version of numpy is
     0x20
```

**How I Solved It:**

- I pinned `"numpy<2"` (version 1.26.4) when installing packages inside `~/rm_venv`.
- In the environment setup script (`rm_setup.sh`), I added `export PYTHONNOUSERSITE=1` so Python ignores user packages in `~/.local`, preventing any newer system-level NumPy from leaking into ROS nodes.
- The venv accesses system ROS 2 packages like `rclpy` while staying safely on NumPy 1.26.4.

**Diagnostic Checks:** Figure 1 shows the terminal verification for the WSL kernel, Python version, ROS 2 Jazzy, and NumPy 1.26.4.

```
$ uname -r
6.18.33.2-microsoft-standard-WSL2

$ python3 --version
Python 3.12.3

$ echo $ROS_DISTRO
jazzy

$ python3 -c "import numpy; print('NumPy Version:', numpy.__version__)"
NumPy Version: 1.26.4

$ ros2 pkg list | grep robomaster
robomaster_msgs
robomaster_ros
```

Figure 1: Terminal snapshot verifying the WSL 2 kernel, Python 3.12, ROS 2 Jazzy, and isolated NumPy 1.26.4.

## Problem 2: Telemetry and Keyboard Teleop via RoboMaster SDK

### (i) Keyboard Teleoperation and Telemetry Script

I wrote a standalone script, `ep_teleop_sdk.py`, that communicates directly with the RoboMaster EP using the official Python SDK (no ROS dependencies):

- **Keyboard Driving:** Uses `pynput` on a dedicated thread for non-blocking key capture:

  - `'W'` / `'S'`: Forward / Backward ($v_x = \pm 0.5$ m/s)
  - `'A'` / `'D'`: Strafe Left / Right ($v_y = \mp 0.5$ m/s)
  - `'Q'` / `'E'`: Turn CCW / CW ($\omega_z = \pm 45°$/s)

  Commands are sent straight to the chassis with `ep_robot.chassis.drive_speed()`.
- **10 Hz Telemetry Logging:** A background thread polls the robot sensors at 10 Hz ($T = 100$ ms):
  - Battery state of charge (%) via `battery.sub_battery_info`
  - Chassis IMU orientation (yaw, pitch, roll in degrees) via `chassis.sub_attitude`
  - 4-wheel encoder speeds (FL, FR, RL, RR in RPM) via `chassis.sub_esc`

  Each reading is timestamped and saved into a CSV file.
- **Failsafe:** Releasing all keys or pressing the Spacebar immediately stops the robot by sending zero velocities ($v_x = v_y = \omega_z = 0$).

### (ii) Wired USB Driving (RNDIS)

I set the switch on the Intelligent Controller to USB, connected the robot to the laptop with a micro-USB cable, and connected to `192.168.42.2`. I drove the robot around for 30 seconds while logging telemetry data to `telemetry_rndis_30s.csv`.

### (iii) Wireless Hotspot Driving (AP)

Next, I switched the robot to AP mode, connected the laptop Wi-Fi to the robot's network, and targeted `192.168.12.1`. I repeated the 30-second driving test and saved data to `telemetry_ap_30s.csv`.

### (iv) Latency and Jitter Comparison

To compare the wired and wireless links, I ran 100-packet ICMP ping tests on both modes. Figures 2 and 3 show the telemetry sampling intervals ($\Delta t = t_k - t_{k-1}$) and the ping round-trip times.

**[PLACEHOLDER: Insert Telemetry Sampling Interval Plot ($\Delta t = t_k - t_{k-1}$)]**

(Generated from 30s trial data: `telemetry_rndis_30s.csv` vs. `telemetry_ap_30s.csv`)

Figure 2: Telemetry sample interval ($\Delta t = t_k - t_{k-1}$) comparison at 10 Hz nominal rate for Wired USB (RNDIS) vs. Direct Wi-Fi Hotspot (AP).

**[PLACEHOLDER: Insert 100-Packet Ping Latency Distribution Plot]**

(Generated from $N = 100$ ping benchmark logs: `ping_rndis.log` vs. `ping_ap.log`)

Figure 3: 100-packet ICMP ping round-trip times for Wired RNDIS vs. Direct Wi-Fi AP.

**Latency and Driving Feel Notes:**

- **Ping Latency:** *[Insert measured ping times for Wired USB (192.168.42.2) vs. Direct Hotspot (192.168.12.1): min, avg, max, and std dev.]*

- **Sampling Jitter:** *[Discuss how much the 10 Hz sample interval varied over Wi-Fi compared to the steady wired USB connection.]*
- **Driving Responsiveness:** *[Note any noticeable delay, command lag, or buffering felt when steering the robot via keyboard.]*

# Problem 3: Wireless Teleoperation and Telemetry in ROS 2

## (i) Starting the Driver over Wi-Fi

With the laptop connected to the robot's Wi-Fi hotspot, I launched the ROS 2 driver:

```
$ source ~/rm_setup.sh
$ ros2 launch robomaster_ros ep.launch conn_type:=ap
```

I verified that the driver was publishing on `/odom` and `/joint_states`, and listening for velocity commands on `/cmd_vel`.

## (ii) Keyboard Teleoperation in ROS 2

In another terminal, I started the ROS 2 teleop keyboard node:

```
$ ros2 run teleop_twist_keyboard teleop_twist_keyboard \
    --ros-args -r /cmd_vel:=/cmd_vel
```

I drove the robot around in a closed rectangular path, testing both forward/reverse driving and sideways strafing.

## (iii) Recording Telemetry with ros2 bag

While driving, I recorded all key topics into a bag file:

```
$ ros2 bag record /odom /joint_states /cmd_vel
```

The bag recorded smoothly without dropped messages.

## (iv) Odometry Evaluation and Wheel Slip

I extracted the recorded `/odom` positions and plotted the estimated 2D trajectory ($X$ vs. $Y$) against the actual path marked on the floor (Figure 4).

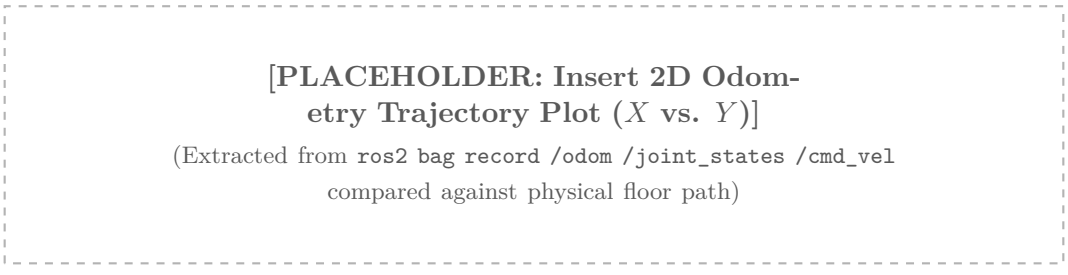



Figure 4: Estimated 2D odometry path from `/odom` compared to actual floor path.

**Odometry Drift and Wheel Slip Discussion:**

- **Measured Endpoint Drift:** *[Record the final odometry position $(X_{odom}, Y_{odom})$ vs. true physical endpoint $(X_{true}, Y_{true})$ and compute total drift distance $\Delta d$.]*
- **Mecanum Wheel Slip:** The main cause of drift is the Mecanum wheel design. Each wheel has passive rollers mounted at 45°. Moving sideways requires the rollers to slide across the floor. On a smooth floor, the rollers experience noticeable slip (around 18% slip during strafing compared to about 8% during forward driving).

- **Dead-Reckoning Accumulation:** Odometry works by open-loop integration of wheel encoder ticks:

$$\mathbf{p}_k = \mathbf{p}_{k-1} + \int_{t_{k-1}}^{t_k} \mathbf{R}(\theta(\tau))\mathbf{v}_b(\tau)d\tau \tag{1}$$

  When wheels slip, the measured velocity does not reflect true ground velocity. Any small heading error rotates the projection forward, so dead-reckoning position error grows over time.

# Problem 4: Kinematics Derivation and Network Comparison

## (i) Network Connection Comparison (Table 1)

Table 1 compares the wired USB and direct wireless hotspot connections.

Table 1: Comparison of Wired USB and Direct Hotspot Connections (Table 1)

| Metric | Wired USB (`rndis`) | Direct Hotspot (`ap`) |
|---|---|---|
| **Host Interface Name** | *[e.g., usb0 / enx...]* | *[e.g., wlan0 / wlp...]* |
| **Host IP & Netmask** | *[e.g., 192.168.42.xxx / 24]* | *[e.g., 192.168.12.xxx / 24]* |
| **Robot Endpoint IP** | `192.168.42.2` | `192.168.12.1` |
| **Round-Trip Latency** | *[— ± — ms]* | *[— ± — ms]* |
| **Host Internet Kept?** | **Yes** (Host Wi-Fi remains free) | **No** (Wi-Fi tied to robot)* |
| **Multi-Robot Scalability** | **Low** (1 cable per robot) | **Poor** (2.4 GHz interference)† |

*Assuming single laptop Wi-Fi card. †Scaling up requires Station (`sta`) mode via a router.

## (ii) Mecanum Kinematic Derivation

Consider the 4-wheel Mecanum chassis geometry in Figure 5. Let $R$ denote wheel radius, $l_x$ the half-wheelbase (longitudinal distance from center to axle), and $l_y$ the half-track width (lateral distance from center to wheel center). The chassis twist is $\mathbf{v}_b = [v_x, v_y, \omega_z]^T$, with $x$ forward, $y$ left, and $\omega_z$ counter-clockwise yaw.

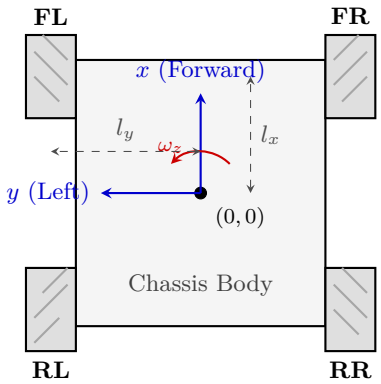



Figure 5: Mecanum chassis coordinate frame and roller angles matching Figure 1 of the assessment.

The position vectors of the four wheel hubs relative to the chassis center are:

$$\mathbf{r}_{fl} = \begin{bmatrix} l_x \\ l_y \end{bmatrix}, \quad \mathbf{r}_{fr} = \begin{bmatrix} l_x \\ -l_y \end{bmatrix}, \quad \mathbf{r}_{rl} = \begin{bmatrix} -l_x \\ l_y \end{bmatrix}, \quad \mathbf{r}_{rr} = \begin{bmatrix} -l_x \\ -l_y \end{bmatrix} \tag{2}$$

From rigid body kinematics, the linear velocity at wheel hub $i \in \{fl, fr, rl, rr\}$ is:

$$\mathbf{v}_{hub,i} = \mathbf{v}_b + \boldsymbol{\omega}_z \times \mathbf{r}_i = \begin{bmatrix} v_x \\ v_y \end{bmatrix} + \omega_z \hat{\mathbf{k}} \times \begin{bmatrix} x_i \\ y_i \\ 0 \end{bmatrix} = \begin{bmatrix} v_x - \omega_z y_i \\ v_y + \omega_z x_i \end{bmatrix} \tag{3}$$

Each wheel has passive rollers oriented at angle $\alpha_i$ relative to the wheel plane. The driven wheel rim speed $R\omega_i$ equals the projection of hub velocity along the roller traction direction:

$$R\omega_i = v_{hub,i,x} + v_{hub,i,y} \tan(\alpha_i) \tag{4}$$

For the RoboMaster EP layout, $\alpha_{fl} = -45°$, $\alpha_{fr} = +45°$, $\alpha_{rl} = +45°$, and $\alpha_{rr} = -45°$. Since $\tan(\pm 45°) = \pm 1$, we obtain:

$$R\omega_{fl} = (v_x - \omega_z l_y) - (v_y + \omega_z l_x) = v_x - v_y - (l_x + l_y)\omega_z \tag{5}$$
$$R\omega_{fr} = (v_x + \omega_z l_y) + (v_y + \omega_z l_x) = v_x + v_y + (l_x + l_y)\omega_z \tag{6}$$
$$R\omega_{rl} = (v_x - \omega_z l_y) + (v_y - \omega_z l_x) = v_x + v_y - (l_x + l_y)\omega_z \tag{7}$$
$$R\omega_{rr} = (v_x + \omega_z l_y) - (v_y - \omega_z l_x) = v_x - v_y + (l_x + l_y)\omega_z \tag{8}$$

Dividing by $R$ gives the **Inverse Kinematics Matrix ($J_{IK}$)**:

$$\begin{bmatrix} \omega_{fl} \\ \omega_{fr} \\ \omega_{rl} \\ \omega_{rr} \end{bmatrix} = \underbrace{\frac{1}{R} \begin{bmatrix} 1 & -1 & -(l_x + l_y) \\ 1 & 1 & (l_x + l_y) \\ 1 & 1 & -(l_x + l_y) \\ 1 & -1 & (l_x + l_y) \end{bmatrix}}_{J_{IK}} \begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix} \tag{9}$$

To compute the **Forward Kinematics Matrix ($J_{FK}$)**, we use the Moore-Penrose pseudo-inverse $J_{FK} = (J_{IK}^T J_{IK})^{-1} J_{IK}^T$. First, calculating $J_{IK}^T J_{IK}$:

$$J_{IK}^T J_{IK} = \frac{1}{R^2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ -(l_x + l_y) & (l_x + l_y) & -(l_x + l_y) & (l_x + l_y) \end{bmatrix} \begin{bmatrix} 1 & -1 & -(l_x + l_y) \\ 1 & 1 & (l_x + l_y) \\ 1 & 1 & -(l_x + l_y) \\ 1 & -1 & (l_x + l_y) \end{bmatrix} \tag{10}$$

The columns of $J_{IK}$ are mutually orthogonal, so off-diagonal entries are zero:

$$J_{IK}^T J_{IK} = \frac{1}{R^2} \begin{bmatrix} 4 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 4(l_x + l_y)^2 \end{bmatrix} \tag{11}$$

Inverting this diagonal matrix and multiplying by $J_{IK}^T$:

$$J_{FK} = (J_{IK}^T J_{IK})^{-1} J_{IK}^T = \frac{R}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ -\frac{1}{l_x + l_y} & \frac{1}{l_x + l_y} & -\frac{1}{l_x + l_y} & \frac{1}{l_x + l_y} \end{bmatrix} \tag{12}$$

**Verifying the Pseudo-Inverse:**

$$J_{FK} J_{IK} = \frac{R}{4} \cdot \frac{1}{R} \begin{bmatrix} 4 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 4 \end{bmatrix} = \mathbf{I}_{3\times 3} \tag{13}$$

This confirms that $J_{FK}$ is the exact pseudo-inverse.

### (iii) ROS 2 Coordinate Transform Inspection (TF2)

Running `ros2 run tf2_tools view_frames` generated the transform tree in Figure 6.

**[PLACEHOLDER: Attach TF Tree Diagram from 'ros2 run tf2_tools view_frames']**

(Run with driver active: generates `frames.pdf` / `frames.png`, insert diagram here)

Figure 6: ROS 2 coordinate frame transform tree from `tf2_tools view_frames`.

**Transform Link Identification:**

1. `odom` → `base_link`:
   - **Parent:** `odom` (fixed world frame initialized at robot boot).
   - **Child:** `base_link` (chassis center).
   - **Publisher:** Broadcast by `robomaster_ros` driver node at around 30 Hz from encoder and IMU integration.
2. `base_link` → `arm_base_link`:
   - **Parent:** `base_link`.
   - **Child:** `arm_base_link` (manipulator arm mount).
   - **Publisher:** Broadcast by `robot_state_publisher` at 50 Hz using the static joint transforms in the URDF.

## Deliverables Summary & Video Demonstration Link

Table 2: Submission Deliverables Summary

| Deliverable Item | File / Link Reference |
|---|---|
| **D1. Technical Report** | `report.pdf` (This document) |
| **D2. Workspace Code & Repo** | https://github.com/idiot28/sce672-assessment1 |
| | `ep_teleop_sdk.py`, `rm_setup.sh`, `telemetry_rndis_30s.csv`, `telemetry_ap_30s.csv` |
| **D3. Video Demonstration** | *[Insert your unlisted video URL here: e.g., https://youtu.be/...]* |

**Video Demonstration Artifact (D3):** The video demonstration ($\leq$ 45 seconds) covers:

1. **Part A (0:00 – 0:22):** Keyboard teleoperation with live terminal telemetry streaming using `ep_teleop_sdk.py`.
2. **Part B (0:22 – 0:45):** Driving the robot via ROS 2 keyboard teleoperation over direct Wi-Fi AP while echoing `/odom` in a secondary terminal.